



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Systems and Methods for Big and Unstructured Data Project

Author(s): **Lorenzo Meroi**
Simone Mauro
Francesco Poloni

Group Number: **41**

Academic Year: 2024-2025

Contents

Contents

1	Mongo DB	1
1.1	Original dataset	1
1.2	Document	3
1.2.1	Example of a document	4
1.3	Data Injection	5
1.4	Queries	7
1.4.1	Exploring our dataset	7
1.4.2	Island	9
1.4.3	Population distribution	12
1.4.4	Most populated areas	13
1.4.5	Which is older, SF or LA?	15
1.4.6	Home Price-to-Income Ratio	16
1.4.7	Most affordable block near SF	19
1.4.8	Average person per household	20
1.4.9	Home price to income ratio	23
1.4.10	Near block with highest difference in median house value	27
2	Neo4j	31
2.1	Dataset Description	31
2.1.1	Data Injection	33
2.1.2	Neo4j Dataset Upload	33
2.2	Queries	35
2.2.1	Queries for Generating Paths	35
2.2.2	Other queries	38
3	Extra	41

3.1 Introduction 41

3.2 Requirements 41

3.3 The notebooks 42

 3.3.1 MongoDB 42

 3.3.2 Neo4j 42

List of Figures 43

List of Tables 45

1 | Mongo DB

1.1. Original dataset

The dataset we used included some indices about the median housing price in a specific latitude and longitude in California from the 1990 California census. Additional data that could impact that price, such as the median income in the area, were also included.

	FIELD	DESCRIPTION
	longitude, latitude	Geographic coordinates specifying the location of block we are taking into account
	housing_median_age	Median age of houses in the area expressed in year
	total_rooms	Total number of rooms in all houses within the block
	total_bedrooms	Total number of bedrooms in all houses within the block
	population	Total population residing in the block
	households	Total number of households in the block
	median_income	Median income of households in the area (in tens of thousands of dollars)
	median_house_value	Median house value in the area (in dollars)
	ocean_proximity	A text value that represents how much we are far from the ocean

Table 1.1: Dataset Variables with Descriptions

The CSV file can be found at the following link: [California Housing Dataset on OpenML](#).

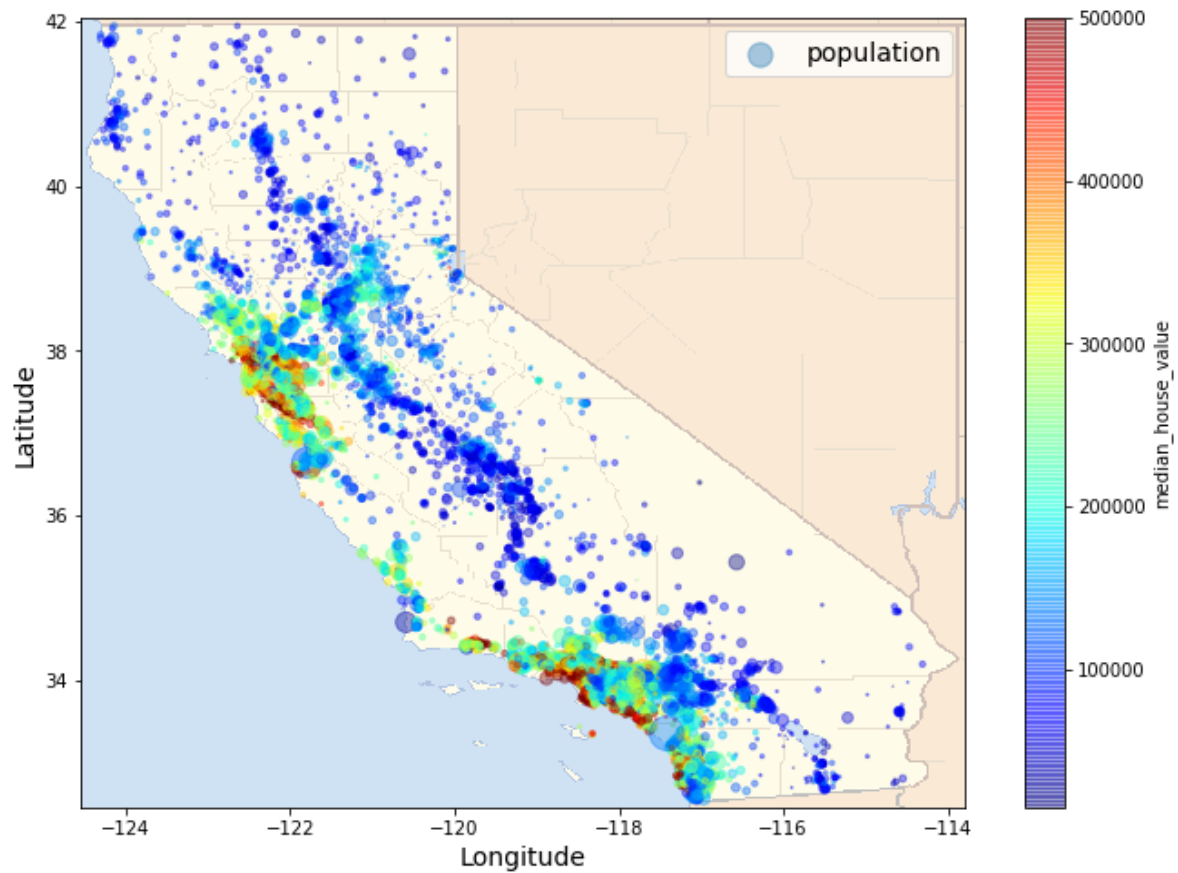
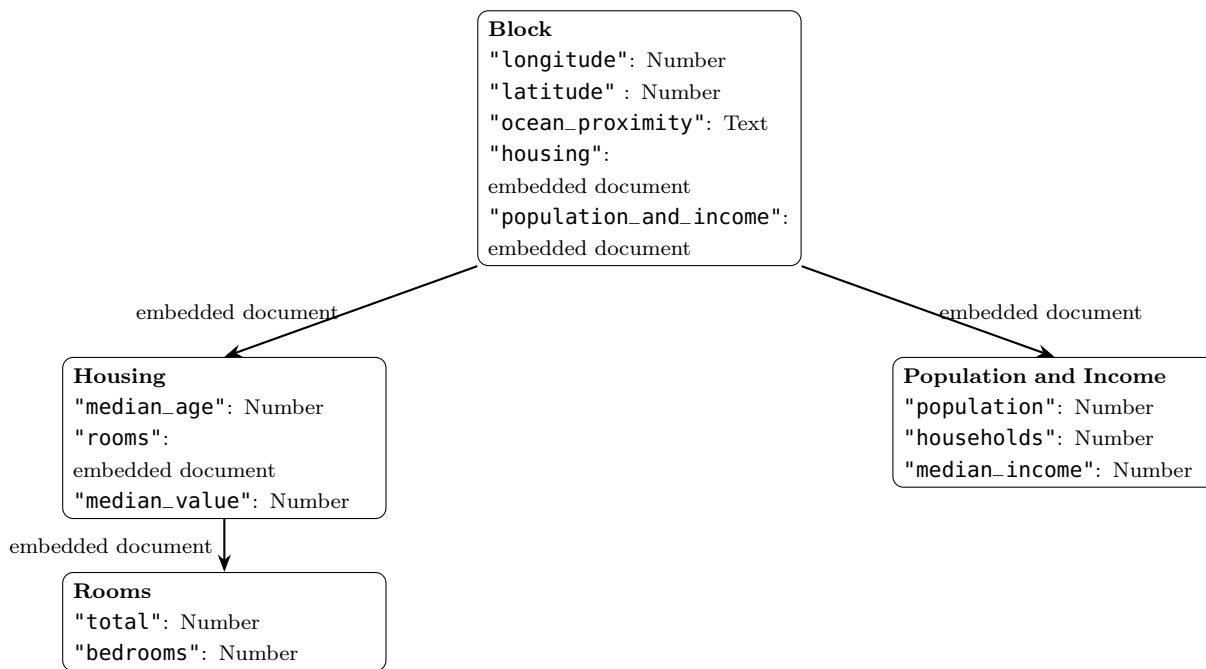


Figure 1.1: Image taken from: California Housing Price Prediction on GitHub

1.2. Document

What you see below is how we organized the data in sub document to have just one simple document in our MongoDB.



1.2.1. Example of a document

This is a document that it is created in our db after we shaped the csv file:

Block:

```

{
  "longitude": -122.05,
  "latitude": 37.37
  "ocean_proximity": "NEAR BAY",
  "housing": {
    "median_age": 27,
    "rooms": {
      "total": 3885,
      "total_bedrooms": 661
    },
    "median_value": 344700
  },
  "population_and_income": {
    "population": 1537,
    "households": 606,
    "median_income": 6.6085
  }
}

```

1.3. Data Injection

The dataset we used was already pretty much cleaned, and complete but we needed to shape it. For obtaining a .json file from the .csv file we had we had to write a Python program that was able to transform and shape the data the way we wanted. Here is the python code we used:

```
1 import pandas as pd
2 import json
3
4 # Load the CSV file
5 csv_file = "PathTSavedCSVfile"
6 df = pd.read_csv(csv_file)
7
8 # 1. Check for missing data
9 print("Missing values per column:\n", df.isnull().sum())
10
11 # 2. Handle missing data
12 # We drop the rows with missing values
13 df = df.dropna()
14
15
16 # 3. Convert to JSON-compatible structure (as per your earlier schema
    )
17 json_data = []
18 for _, row in df.iterrows():
19     document = {
20         "longitude": row["longitude"],
21         "latitude": row["latitude"],
22         "ocean_proximity": row["ocean_proximity"],
23         "housing": {
24             "median_age": row["housing_median_age"],
25             "rooms": {
26                 "total": row["total_rooms"],
27                 "bedrooms": row["total_bedrooms"]
28             },
29             "median_value": row["median_house_value"]
30         },
```

```

31     "population_and_income": {
32         "population": row["population"],
33         "households": row["households"],
34         "median_income": row["median_income"]
35     }
36 }
37 json_data.append(document)
38
39 # 4. Save cleaned JSON
40 output_file = "California_Housing.json"
41 with open(output_file, "w") as f:
42     json.dump(json_data, f, indent=4)
43
44 print(f"CleaneD JSON file saved to {output_file}")

```

We then proceeded to create a db and a collection inside it using compass, a graphical tool studied for Mongo DB, to help us:

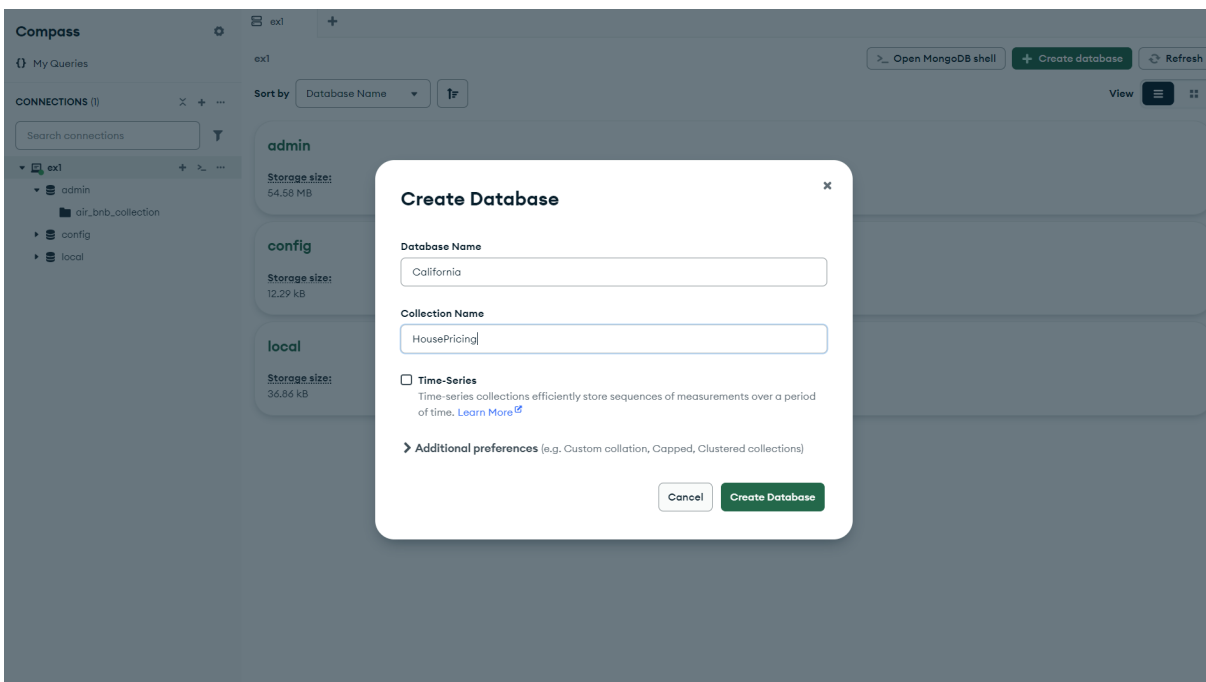


Figure 1.2: Db and collection creation

1.4. Queries

In this section we will see the query we have done and why we thought they were interesting. This section is divided into subsection that represents either a query or a set of query done to answer a question.

1.4.1. Exploring our dataset

Problem regarding the id:

We grouped these types of queries into a single category because, although they are not very challenging, they provide crucial information about our dataset. The website we obtained the dataset from explicitly mentioned that data cleaning would be necessary. Additionally, as described in section 1.1, the website did not provide detailed information on the dataset, prompting us to execute several queries to better understand its structure. At first, we assigned the `_id` in MongoDB to the `latitude` and `longitude` coordinates. However, although Compass imported the database without any issues, the logs showed errors. Upon further investigation, we discovered that the problem was likely due to non-unique coordinate values for certain blocks. Subsequently, we altered the database architecture by utilizing the `_id` automatically generated by Mongo. Following this adjustment, the errors ceased. To verify our hypothesis regarding the cause, we executed a query to group entries by latitude and longitude. This query showed that some blocks are so closely located that they are detected to have identical latitude and longitude coordinates.

```
1 db.HousePricing.aggregate([
2   {
3     $group: {
4       _id: { latitude: "$latitude", longitude: "$longitude" },
5       count: { $sum: 1 }
6     }
7   },
8   {
9     $match: {
10       count: { $gt: 1 }
11     }
12   },
13   { $sort: {count :- 1} }
14 ])
```

```
15 ]);
```

the result is the one we expected:

```
{
  "location": {
    "latitude": 37.8,
    "longitude": -122.41
  },
  "count": 15
}
```

```
{
  "location": {
    "latitude": 37.78,
    "longitude": -122.44
  },
  "count": 11
} ...
```

Ocean proximity values

A subsequent investigation can be conducted to determine the categories available for ocean proximity in our databases, as the website from which we downloaded the db did not provide this information, which is essential to clarify before commencing the main analysis.

```
1 db.HousePricing. aggregate([
2   {$group: {
3     _id: "$ocean_proximity",
4   }
5 }
6 1);
```

Result:

```
{
  "ocean_proximity": "INLAND"
}
```

```
{  
  "ocean_proximity": "NEAR BAY"  
}
```

```
{  
  "ocean_proximity": "NEAR OCEAN"  
}
```

```
{  
  "ocean_proximity": "ISLAND"  
}
```

```
{  
  "ocean_proximity": "<1H OCEAN"  
}
```

Housing price capped at 501000\$

The final straightforward query we conducted aimed to verify our suspicion that house prices are limited to 501000\$. To confirm this, we executed the following query:

```
1 db.HousePricing.countDocuments({  
2   "housing.median_value": { $gt: 500001 }  
3 });
```

and the return was 0 while for the same query with value equal to 500001 the value was 958. This is important because it means that when doing statistic on this we need to remember to remove the richest houses.

1.4.2. Island

Although we are enthusiasts of California, we were unaware of the existence of any populated islands. This piqued our curiosity, leading us to investigate the number of blocks on the island by executing this query:

```
1 db.HousePricing.countDocuments({  
2   "ocean_proximity": "ISLAND"  
3 });
```

The outcome is 5. We became intrigued to determine whether they all originated from the same island, prompting us to execute this query:

```
1 db.HousePricing.find({ocean_proximity:"ISLAND"}, {_id : 0, longitude: 1, latitude:1})
```

and found that all values of longitude and latitude that are very close to each other:

```
{
  "longitude": -118.32,
  "latitude": 33.35
}
```

```
{
  "longitude": -118.33,
  "latitude": 33.34
}
```

```
{
  "longitude": -118.32,
  "latitude": 33.33
}
```

```
{
  "longitude": -118.32,
  "latitude": 33.34
}
```

```
{
  "longitude": -118.48,
  "latitude": 33.43
}
```

We concluded that they are all part of the same big inhabited island. The actual island is called Santa Catalina and it is one of the few inhabited islands of California.



Figure 1.3: Avalon, Santa Catalina Island

We were interested in comparing the median house value on this island to those throughout the rest of California, so we examined this:

```
1 db.HousePricing.aggregate([
2   {$match: {ocean_proximity: "ISLAND" }},
3   {
4     $group: {
5       _id: null,
6       median_house_value: { $avg: "$housing.median_value" }
7     }
8   }
9 ])
```

this returned the value 380440, which is much higher than the general one of California we get by removing the first constraint:

```
1 db.HousePricing.aggregate([
2   {
3     $group: {
4       _id: null,
5       median_house_value: { $avg: "$housing.median_value" }
6     }
7   }
8 ])
```

```
7   }  
8 ])
```

The result obtained from this query is roughly 206864. This aligns with our expectations, as San Catalina appears to be a truly beautiful island from the photos.

1.4.3. Population distribution

We were curious to see about the distribution of the population there. Therefore, we executed a query to retrieve the 1000 most densely populated blocks, categorizing them by their `ocean_proximity` values. This allows us to verify whether the densest areas in California are indeed located near the coast, as we assumed based on our limited knowledge of the state.:

```
1 db.HousePricing.aggregate([  
2   {$sort : { population : -1}},  
3   {$limit : 1000},  
4   {$group : {_id : "$ocean_proximity" , population : {$sum : "  
5     $population_and_income. population"}} }  
6 ]);
```

with this query we got these results:

```
{  
  "ocean_proximity": "<1H OCEAN",  
  "population_and_income": {  
    "population": 164759  
  }  
}
```

```
{  
  "ocean_proximity": "INLAND",  
  "population_and_income": {  
    "population": 58631  
  }  
}
```

```
{
```

```

"ocean_proximity": "NEAR BAY",
"population_and_income": {
  "population": 998872
}
}

```

This is the result we were expecting, knowing there are a lot of big cities in California near the ocean:

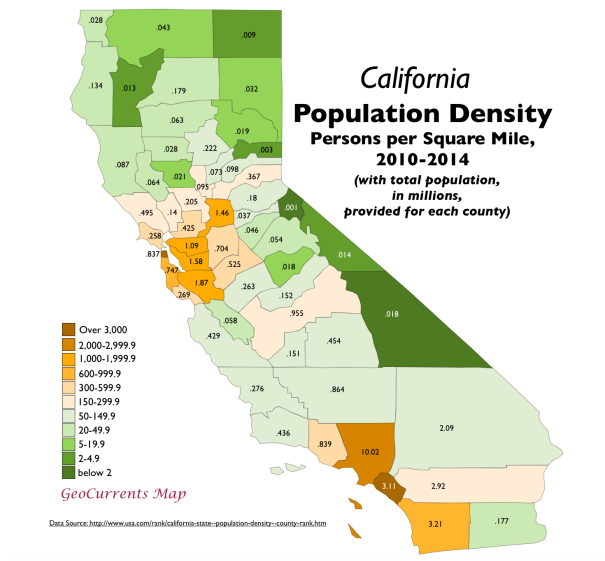


Figure 1.4: Image taken from: California population distribution

1.4.4. Most populated areas

Now we are interested in retrieving the areas/cities that are actually the most populated. To do so, we will group each block by the longitude and latitude with an integer approximation. This is the resulting query:

```

1 db.HousePricing.aggregate([
2   {
3     $group: {
4       _id: {
5         longitude: { $round: ["$longitude", 0] },
6         latitude: { $round: ["$latitude", 0] }
7       },
8       totalPopulation: { $sum: "$population_and_income.
9         population" },
    }
  ]

```

```
10     },
11     {
12         $sort: { totalPopulation: -1 }
13     },
14     {
15         $limit: 4
16     }
17 ]);
```

with results:

```
{
  "location": {
    "longitude": -118,
    "latitude": 34
  },
  "totalPopulation": 10685969
}
```

```
{
  "location": {
    "longitude": -122,
    "latitude": 38
  },
  "totalPopulation": 3690156
}
```

```
{
  "location": {
    "longitude": -117,
    "latitude": 33
  },
  "totalPopulation": 2433763
}
```

```
{
  "location": {
```

```
"longitude": -122,  
"latitude": 37  
},  
"totalPopulation": 2181651  
}
```

Upon examining the results, we identified the coordinates for the most densely populated cities in California. The first coordinate corresponds to Los Angeles, followed by the second for San Francisco, the third for San Diego, and the fourth for San Jose.



Figure 1.5: Los Angeles, Hollywood

1.4.5. Which is older, SF or LA?

We have always seen San Francisco (SF) and Los Angeles (LA) as modern cities where houses are all relatively new so we were curious to see between the 2 which contained the oldest housing according to our db. So we run these 2 queries:

```
1 db.HousePricing.aggregate([  
2   {  
3     $match: {  
4       longitude: { $gte: -118.5 , $lte: -117.5 },  
5       latitude: { $gte: 33.5, $lte: 34.5 }  
6     }  
7   },
```

```

8   {
9     $group: {
10      _id: "Los Angeles",
11      averageMedianAge: { $avg: "$housing.median_age" }
12    }
13  }
14 ]);

```

and

```

1  db.HousePricing.aggregate([
2    {
3      $match: {
4        longitude: { $gte: -122.5, $lte: -121.4 },
5        latitude: { $gte: 37.5, $lte: 38.4 }
6      }
7    },
8    {
9      $group: {
10       _id: "San Francisco",
11       averageMedianAge: { $avg: "$housing.median_age" }
12     }
13   }
14 ]);

```

finding the result that the 2 cities and suburbs are basically infrastructurally modern the same:

```

{
  "location": "Los Angeles",
  "averageMedianAge": 31.760942291608586
} {
  "location": "San Francisco",
  "averageMedianAge": 35.27605834222697
}

```

1.4.6. Home Price-to-Income Ratio

Now we were curious to compare the 2 most renown city of California for the median income compared to the median house value. Although the median house value does not

represent exactly the cost of living for this kind of database it is the best we can do. So we run these 2 queries:

```
1 db.HousePricing.aggregate([
2   {
3     $match: {
4       longitude: { $gte: -118.5 , $lte: -117.5 },
5       latitude: { $gte: 33.5, $lte: 34.5 }
6     }
7   },
8   {
9     $group: {
10      _id: "Los Angeles",
11      averageIncomeToLivingCostRatio: {
12        $avg: {
13          $divide: ["$population_and_income.median_income",
14                  "$housing.median_value"]
15        }
16      }
17    }
18  ]);
```

and

```
1 db.HousePricing.aggregate([
2   {
3     $match: {
4       "longitude": { $gte: -122.5, $lte: -121.5 },
5       "latitude": { $gte: 37.5, $lte: 38.5 }
6     }
7   },
8   {
9     $group: {
10      _id: "San Francisco",
11      averageIncomeToLivingCostRatio: {
12        $avg: {
13          $divide: ["$population_and_income.median_income",
14                  "$housing.median_value"]
15        }
16      }
17    }
18  ]);
```

```
16     }  
17   }  
18  });
```

and found these results: {

```
"location": "Los Angeles",  
"averageIncomeToLivingCostRatio": 0.000018154235715803063  
} {  
"location": "San Francisco",  
"averageIncomeToLivingCostRatio": 0.000018695463849772272  
}
```

In this case we found a significant difference in this ratio telling us that the most affordable city is actually SF.



Figure 1.6: San Francisco

The fact that San Francisco has an higher Home Price-to-Income Ratio is also confirmed also in this study and it is also true today: [link](#).

1.4.7. Most affordable block near SF

Since as good computer scientist we always dream about living in the Silicon Valley we were curious also to see what was the most affordable block near San Francisco so we run this query:

```
1 db.HousePricing.aggregate([
2   {
3     $match: {
4       "longitude": { $gte: -122.5, $lte: -121.5 },
5       "latitude": { $gte: 37.5, $lte: 38.5 }
6     }
7   },
8   {
9     $sort: { "housing.median_value": 1 } // Sort by median value
10    in ascending order
11  },
12  {
13    $limit: 1 // Take the first block with the lowest median
14    value
15  },
16  {
17    $project: {
18      _id: 1,
19      "longitude": 1,
20      "latitude": 1,
21      "housing.median_value": 1
22    }
23  }
24 ]);
```

And found this result:

```
{
  "_id": "ObjectId('674e07b554da08875740ddd2')",
  "longitude": -122.32,
  "latitude": 37.93,
  "housing": {
    "median_value": 22500
  }
}
```

This area is not actually in the center of San Francisco but it is not even so far. From the center of SF we it is approximately 30 minutes.

1.4.8. Average person per household

Now we were curious to have some data about the person per household ratio querying the avg of this ratio and the std deviation in the various block to know more or less how numerous are the families there:

```

1 db.HousePricing.aggregate([
2   {
3     $project: {
4       person_per_household: {
5         $cond: [
6           { $ne: ["$population_and_income.households", 0]
7             }, // Avoid division by zero
8           { $divide: ["$population_and_income.population",
9             "$population_and_income.households"] },
10          null // If households is 0, set ratio to null
11        ]
12      }
13    },
14    {
15      $match: {
16        person_per_household: { $ne: null } // Exclude null
17        ratios (e.g., where households = 0)
18      }
19    },
20    {
21      $group: {
22        _id: null,
23        avgRatio: { $avg: "$person_per_household" },
24        stdDevRatio: { $stdDevPop: "$person_per_household" }
25      }
26    }
27  ]);

```

this is his result: **Block:**

```
{
```

```
"_id": null,  
"avgRatio": 3.071532851191042,  
"stdDevRatio": 10.438013767697397  
}
```

We were shocked by the enormous standard deviation so we decided to investigate further with the following query aiming to discover more about the outliers:

```
1 db.HousePricing.aggregate([  
2   {  
3     $project: {  
4       person_per_household: {  
5         $divide: ["$population_and_income.population", "  
6           $population_and_income.households"]  
7       }  
8   },  
9   {  
10    $match: {  
11      person_per_household: { $gt: 20 }  
12    }  
13  },  
14 ]);
```

And we discovered that there are outliers data like these ones: {

```
"_id": "ObjectId('674e07b554da08875740e3c6')",  
"person_per_household": 599.7142857142857  
} ... {  
"_id": "ObjectId('674e07b654da088757412045')",  
"person_per_household": 1243.3333333333333  
}
```

So we decided to make a query like the first one but eliminating the outliers:

```
1 db.HousePricing.aggregate([  
2   {  
3     $project: {  
4       person_per_household: {  
5         $divide: ["$population_and_income.population", "  
6           $population_and_income.households"]  
7       }  
8   }  
9 ]);
```

```

7      }
8    },
9    {
10     $match: {
11       person_per_household: { $lte: 20 } // Filter out extreme
           ratios
12     }
13   },
14   {
15     $group: {
16       _id: null,
17       avgPersonPerHousehold: { $avg: "$person_per_household" },
18       stdDevPersonPerHousehold: { $stdDevPop: "
           $person_per_household" }
19     }
20   }
21 ]});

```

Block:

```

{
  "_id": null,
  "avgPersonPerHousehold": 2.932484316128325,
  "stdDevPersonPerHousehold": 0.8698095598200718
}

```

This standard deviation actually makes a lot more sense to us now.

Now we were curious to see why we had this extreme outliers so we run the following easy queries:

```

1 db.HousePricing.find({ _id: ObjectId('674e07b654da088757412045') }, {
    longitude:1, latitude:1})

```

```

return: {
  "_id": "ObjectId('674e07b554da08875740e3c6')",
  "longitude": -120.51,
  "latitude": 40.41
}

```

```

1 db.HousePricing.find({ _id: ObjectId('674e07b654da088757412045') }, {
    longitude:1, latitude:1})

```

```
{
  "_id": "ObjectId('674e07b654da088757412045')",
  "longitude": -121.98,
  "latitude": 38.32
}
```

Now if we investigate further using a website like <https://gps-coordinates.org/> to know what area we are looking at we notice that all of these geographical coordinate correspond to prisons. The first one is High Desert State Prison while the second is California State Prison Solano. So probably they considered the prisoners as a single household.



Figure 1.7: High desert state prison

1.4.9. Home price to income ratio

We know well that Americans often spend beyond their means. So we thought that an interesting query would be to know the area in California where people spend the most, relative to their income, on housing. To do so we exploit the trick we used previously to group part of America approximating the latitude and the longitude to one digit:

```
1 db.HousePricing.aggregate([
2   {
3     $group: {
4       _id: {
5         longitude: { $round: ["$longitude", 0] },
6         latitude: { $round: ["$latitude", 0] }
```

```

7      },
8      homeprice_to_income: {
9          $avg: {
10             $divide: ["$housing.median_value", "
11                        $population_and_income.median_income"]
12             }
13         }
14     },
15     {
16         $sort: { homeprice_to_income: -1 }
17     },
18     {
19 $limit:1
20 }
21 ]);

```

and this is the result:

```

{
  "_id": {
    "longitude": -120,
    "latitude": 34
  },
  "homeprice_to_income": 88376.19072601027
}

```

This points directly to the complex of islands near the bay. At first glance someone could think that are the housing in island. actually if we ran this query:

```

1 db.HousePricing.aggregate([
2     {
3         $match: {
4             "longitude": { $gte: -120.4, $lte: -119.6 },
5             "latitude": { $gte: 33.6, $lte: 34.4 },
6             "ocean_proximity": "ISLAND"
7         }
8     }
9 ]);

```

we don't find any results, and rightfully so since the 2 island are natural reserve. To know so why there is a so high diproportion there we could run this other query:

```
1 db.HousePricing.aggregate([
2   {
3     $match: {
4       "longitude": { $gte: -120.4, $lte: -119.6 },
5       "latitude": { $gte: 33.6, $lte: 34.4 }
6     }
7   },
8   {
9     $sort: { "population_and_income.median_income": 1 }
10  },
11  {
12    $project: {
13      _id: 0, // Exclude the default _id field
14      longitude: 1,
15      latitude: 1,
16      "housing.median_value": 1,
17      "population_and_income.median_income": 1
18    }
19  }
20 ]);
```

this will return **Block:**

```
{
  "longitude": -119.86,
  "latitude": 34.38,
  "housing": {
    "median_value": 316700
  },
  "population_and_income": {
    "median_income": 1.5071
  }
}
```

Block:

```
{
  "longitude": -119.85,
```

```
"latitude": 34.4,  
"housing": {  
  "median_value": 37500  
},  
"population_and_income": {  
  "median_income": 2.1652  
}  
}
```

Block:

```
{  
  "longitude": -119.69,  
  "latitude": 34.38,  
  "housing": {  
    "median_value": 281300  
  },  
  "population_and_income": {  
    "median_income": 2.25  
  }  
}
```

That are all coordinate of Santa Barbara. The high ratio is due to the fact that Santa Barbara, is one of the most turistic cities of the Bay but does not have big tech company like San Francisco. This lead to a big disproportion to house value and actual income.



Figure 1.8: Santa Barbara

1.4.10. Near block with highest difference in median house value

The last query we found interesting was a query to return the 2 blocks in San Francisco which are near but have the highest difference in median house value. To simplify things we say that to be near the 2 blocks must have a difference of longitude and latitude which is ≤ 0.01 . So we run this difficult query:

```
1 db.HousePricing.aggregate([
2   // Step 1: Filter for San Francisco area
3   {
4     $match: {
5       longitude: { $gte: -123, $lte: -122 },
6       latitude: { $gte: 37, $lte: 38 }
7     }
8   },
9   // Step 2: Self-join to find nearby blocks
10  {
11    $lookup: {
12      from: "HousePricing",
13      as: "neighbors",
14      let: { currentId: "$_id", currentLongitude: "$longitude",
              currentLatitude: "$latitude", currentMedianValue: "$housing.median_value" },
```

```

15     pipeline: [
16         {
17             $match: {
18                 $expr: {
19                     $and: [
20                         { $lte: [{ $abs: { $subtract: ["
21                             $longitude", "$$currentLongitude"]
22                             } }, 0.01] },
23                         { $lte: [{ $abs: { $subtract: ["
24                             $latitude", "$$currentLatitude"] }
25                             }, 0.01] },
26                         { $ne: ["$_id", "$$currentId"] } //
27                             Ensure different blocks
28                     ]
29                 }
30             },
31             {
32                 $project: {
33                     _id: 0,
34                     longitude: 1,
35                     latitude: 1,
36                     neighborMedianValue: "$housing.median_value"
37                 }
38             }
39         ]
40     },
41     // Step 3: Flatten the results to get pairs of blocks
42     {
43         $unwind: "$neighbors"
44     },
45     // Step 4: Compute the absolute difference in median house value
46     {
47         $addFields: {
48             differenceInMedianValue: {
49                 $abs: { $subtract: ["$housing.median_value", "
50                     $neighbors.neighborMedianValue"] }
51             }
52         }
53     }
54 ]

```

```
48     }
49   },
50   // Step 5: Sort by the difference in descending order
51   {
52     $sort: { differenceInMedianValue: -1 }
53   },
54   // Step 6: Limit to get just 1 result.
55   {
56     $limit: 1
57   },
58   // Step 7: Project the final output
59   {
60     $project: {
61       _id: 0,
62       block1: {
63         longitude: "$longitude",
64         latitude: "$latitude",
65         medianValue: "$housing.median_value"
66       },
67       block2: {
68         longitude: "$neighbors.longitude",
69         latitude: "$neighbors.latitude",
70         medianValue: "$neighbors.neighborMedianValue"
71       },
72       differenceInMedianValue: 1
73     }
74   }
75 ];
```

and we find these results:

```
{
  "differenceInMedianValue": 467501,
  "block1": {
    "longitude": -122.41,
    "latitude": 37.78,
    "medianValue": 32500
  },
  "block2": {
    "longitude": -122.41,
```

```
"latitude": 37.79,  
"medianValue": 500001  
}  
}
```

As we can see on google maps the richest area actually correspond to one of the most renown square of San Francisco, Union square. In Union square there are not actually not so many housing (being that area mostly dedicated to expensive hotel and shops) but the one we have are all over the max limit of 501000\$. The other more periferic area instead is probably corresponding to China Town, that is famous instead for the organized crime especially in the 90s, so maybe the low price is due to that.

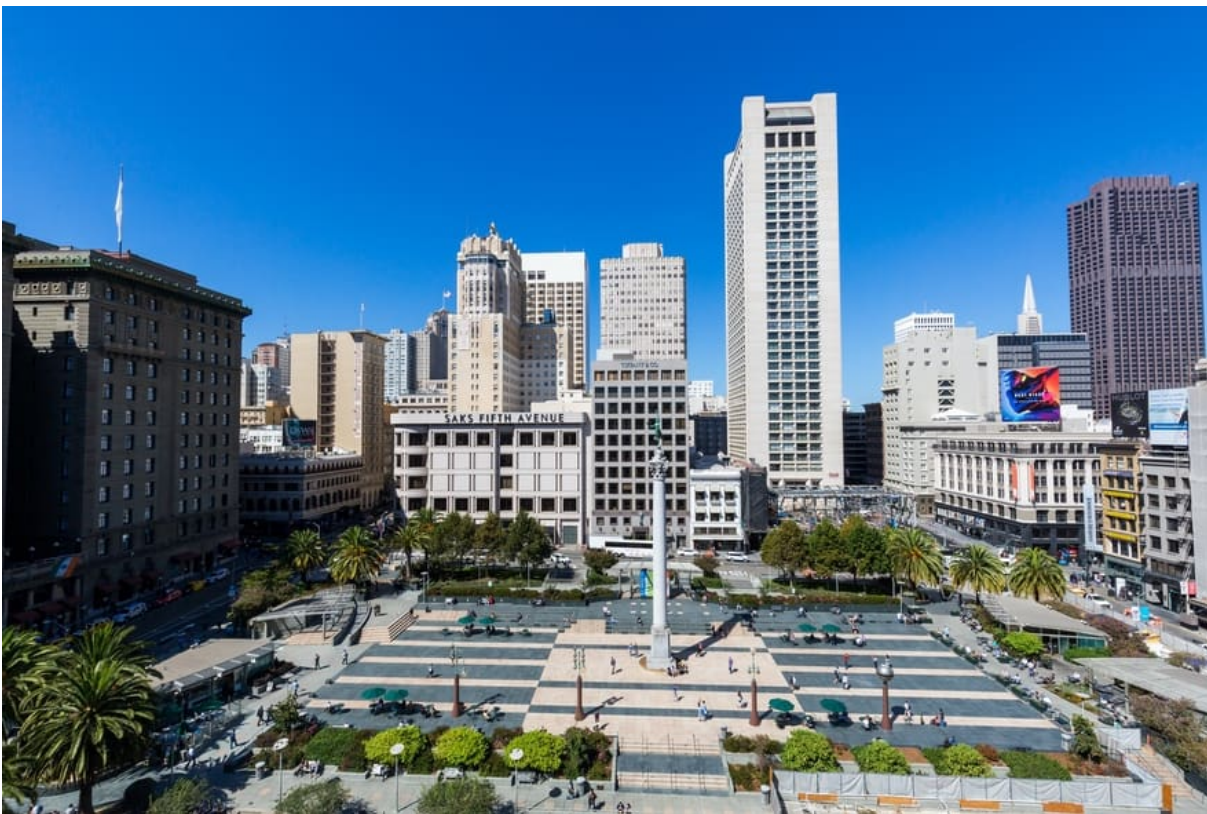


Figure 1.9: Union Square

2 | Neo4j

2.1. Dataset Description

The original dataset comes from this github page and the London Datastore website. The data was provided in csv, with some cypher scripts to load only a small sample of the data corresponding to the metro lines, we then had to clean the bus network data and write our own scripts for it. This data represents the London transport network from 2010, it is composed of stations, bus stops and the connections between them. The structure is a graph with:

- **Nodes:** Representing metro/train stations and bus stops.
- **Relationships:** Representing connections between these nodes based on transport lines and bus routes.

Nodes

Stations

- Represents metro/train stations.
- Key properties:
 - **zone:** Fare zone of the station (a number between 1 and 3).
 - **latitude, longitude:** Geographic coordinates.
 - **name:** Name of the station.

Bus Stops

- Represents bus stops in the transport network.
- Key properties:
 - **bus_stop_code:** Unique identifier for the stop.

- **latitude, longitude**: Geographic coordinates.
- **name**: Name of the bus stop.
- **stop_area**: Area identifier grouping related stops.
- **stop_Code_LBSL**: Unique code assigned to the stop by London Bus Services Limited.
- **naptan_ATCO_code**: Association of Transport Coordinating Officers code according to the UK's National Public Transport Access Nodes system.
- **virtual_stop**: Indicates if the stop is virtual (1 for virtual, 0 for physical).

Relationships

CONNECTED_BY

- Connects two **Station** nodes.
- Key property:
 - **line**: The transport line (e.g. "TFL RAIL") connecting the stations.

NEXT_STOP

- Connects two **BusStop** nodes.
- Key properties:
 - **route**: The bus route number.
 - **heading**: Directional information for the route.
 - **run**: A number that indicates different variations of the route.
 - **virtual**: Specifies if the connection involves a virtual stop.

Key Characteristics

- The dataset is geospatially aware, with **latitude** and **longitude** available for each node, enabling geographic computations such as distances and nearest neighbors..
- Metro stations and bus stops are **not directly connected** in the graph, as they represent distinct transport layers.

2.1.1. Data Injection

we used this python script to add a next stop column in the "london_transport_datasets_routes.csv" we uploaded the processed dataset in the "london_transport_datasets_routes_processed.csv" file, this is used later to create the NEXT_STOP relation in neo4j.

```

1     import pandas as pd
2     df = pd.read_csv('london_transport_datasets_routes.csv')
3
4     # Sort the values
5     df = df.sort_values(by=['Route', 'Run', 'Sequence'])
6
7     # Add a colon for the next stop
8     df['Next_Stop_Code_LBSL'] = df.groupby(['Route', 'Run'])['
        Stop_Code_LBSL'].shift(-1)
9
10    df.to_csv('london_transport_datasets_routes_processed.csv', index=
        False)

```

]

The "london_transport_datasets_routes_processed.csv" file was still quite big so we decided to split it in multiple csv files. This allowed us to import the dataset in smaller steps, in this specific use case we decided to split it in 6 files composed of 10.000 lines each.

2.1.2. Neo4j Dataset Upload

In this section it is reported the list of Cypher commands used to extract data from CSVs and import it into Neo4j. **For quicker installation, we highly recommend using the database dump that we have provided instead.**

This command loads the nodes that represent the Metro Stations

```

1     LOAD CSV WITH HEADERS FROM "file:///
        london_transport_datasets_London_stations.csv"
2     AS row
3     MERGE (s:Station {latitude:toFloat(row.Latitude), longitude:toFloat(
        row.Longitude), name:row.Station, zone:row.Zone})

```

This command connects the stations and labels the connections according to transit lines they represent.

```

1 LOAD CSV WITH HEADERS FROM "file:///
    london_transport_datasets_London_tube_lines.csv" AS row
2 MATCH (a:Station {name: row.From_Station}), (b:Station {name: row.
    To_Station})
3 MERGE (a)-[rel1:CONNECTED_BY {line: toUpper(row.Tube_Line)}]->(b)
4 MERGE (b)-[rel2:CONNECTED_BY {line: toUpper(row.Tube_Line)}]->(a)
5 RETURN rel1, rel2;

```

This command loads the nodes that represent the Bus Stops. We found 31 virtual stop with some null values, we decided to replace these values with “Unknown”.

```

1 LOAD CSV WITH HEADERS FROM "file:///london_transport_datasets_stops.
    csv" AS row
2 MERGE (s:BusStop {
3     stop_Code_LBSL: coalesce(row.Stop_Code_LBSL, 'Unknown'),
4     bus_stop_code: coalesce(row.Bus_Stop_Code, 'Unknown'),
5     naptan_ATCO_code: coalesce(row.Naptan_Atco, 'Unknown'),
6     name: coalesce(row.Stop_Name, 'Unknown'),
7     stop_area: coalesce(row.Stop_Area, 'Unknown'),
8     virtual_stop: coalesce(row.Virtual_Bus_Stop, 'Unknown'),
9     latitude: coalesce(toFloat(row.Latitude), 0.0),
10    longitude: coalesce(toFloat(row.Longitude), 0.0)
11 })

```

This command connects the bus stops and labels the connections according to bus lines they represent. Some heading values were null, we decided to replace these values with “Unknown”.

The “london_transport_datasets_routes_processed.csv” file is quite big, the following query loads the whole csv file, you can replace “file:///london_transport_datasets_routes_processed.csv” with “file:///london_transport_datasets_routes_processed_part1.csv” and repeat the query for each file from part1 to part6.

```

1 LOAD CSV WITH HEADERS FROM "file:///
    london_transport_datasets_routes_processed.csv" AS row
2 MATCH (start:BusStop {stop_Code_LBSL: row.Stop_Code_LBSL})
3 MATCH (end:BusStop {stop_Code_LBSL: row.Next_Stop_Code_LBSL})
4 MERGE (start)-[rel:NEXT_STOP]->(end)
5 ON CREATE SET

```

```

6     rel.route = row.Route,
7     rel.run = row.Run,
8     rel.heading = COALESCE(row.Heading, "Unknown"),
9     rel.virtual = row.Virtual_Bus_Stop;

```

2.2. Queries

2.2.1. Queries for Generating Paths

Let us start with some queries that allow us to visualise the data like in figure below.

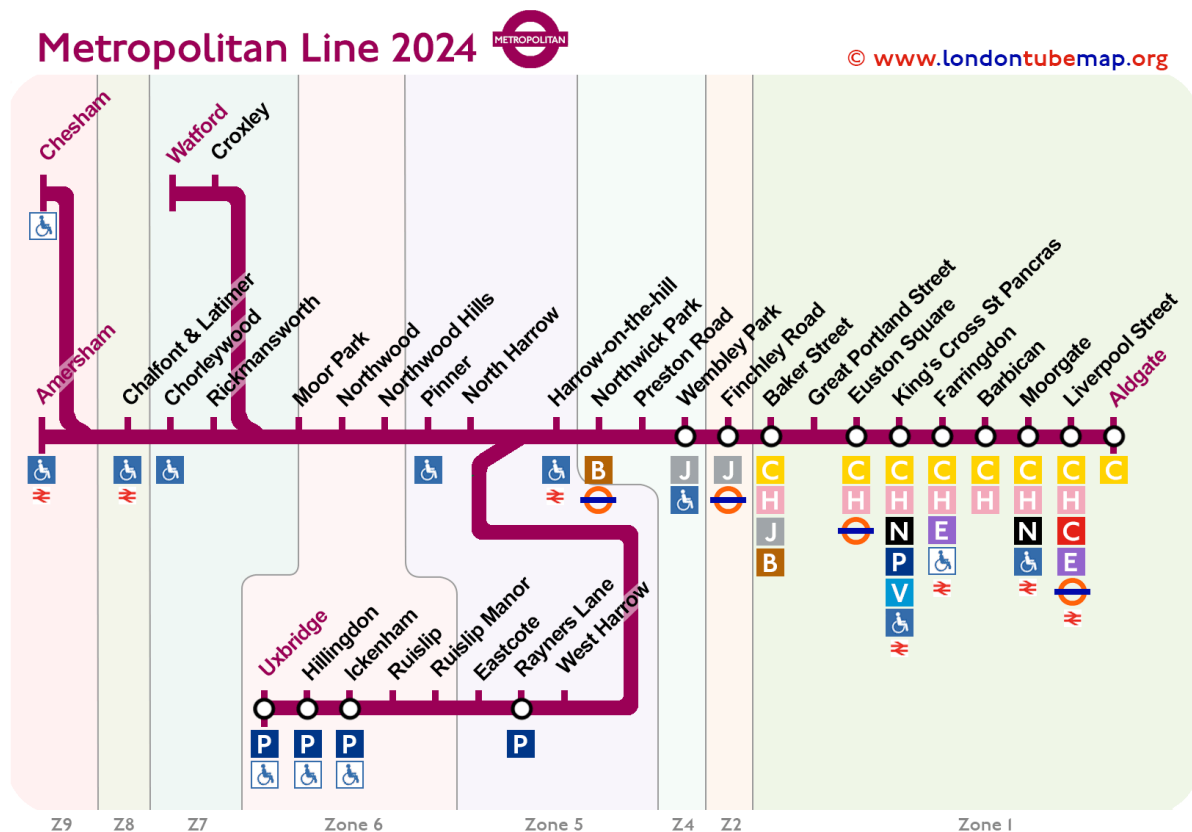


Figure 2.1: Metropolitan line

The first one that comes to mind is a query that allows us to see all the stations in a train line. We can do this with this simple match that searches all the nodes connected to a line. This is a very simple query but probably one of the most useful ones so we decided to include it anyway:

```

1 MATCH (start:Station)-[:CONNECTED_BY {line: "METROPOLITAN"}]-(end:
   Station)
2 RETURN DISTINCT start.name

```

This is the same query but for the bus network:

```

1 MATCH (start:BusStop)-[:NEXT_STOP {route: "154"}]-(end:BusStop)
2 RETURN DISTINCT start.name AS BusStop

```

If we want to visualise the nodes in Neo4j Browser, we can simply replace "DISTINCT start.name" with start.

If we can't visualize the graph generated from Neo4j Browser it can be hard to understand the available paths. The following query is a variation of the previous query that returns all the paths from the terminal stations.

This is done by filtering the start and end nodes by the number of connection and by eliminating all the paths where a node appears more than one time. The query returns the path only in one direction (from A to B and not from B to A, where A has a lower id number):

```

1 MATCH path = (start:Station)-[:CONNECTED_BY* {line: "METROPOLITAN"
   }]->(end:Station)
2 WHERE size([(start)-[:CONNECTED_BY {line: "METROPOLITAN"}]-( ) | 1]) =
   2
3 AND size([(end)-[:CONNECTED_BY {line: "METROPOLITAN"}]-( ) | 1]) = 2
4 AND ALL(n IN nodes(path) WHERE SINGLE(x IN nodes(path) WHERE x = n)
   )
5 AND id(start) < id(end)
6 RETURN
7 [node IN nodes(path) | node.name] AS Route,
8 length(path) AS RouteLength
9 ORDER BY RouteLength DESC

```

Another useful feature is the ability to search for the reachable stations or bus stops from a specific position. The following query returns all the stations or bus stops reachable in 3 steps. In this specific case it returns all the station names and path length from the "Stratford" station :

```

1 MATCH path = (start:Station {name: "Stratford"})-[:CONNECTED_BY
   *..3]-(reachable)
2 RETURN DISTINCT
3   reachable.name AS ReachableNode,
4   length(path) AS PathLength
5 ORDER BY PathLength ASC

```

This is the same query but with the bus stops:

```

1 MATCH path=(start:BusStop {name: "WHITGIFT CENTRE"})-[:NEXT_STOP
   *..3]-(reachable)
2 RETURN DISTINCT
3   reachable.name AS ReachableNode,
4   length(path) AS PathLength
5 ORDER BY PathLength ASC

```

The following query allows us to see how connected are all the stations, it returns the number of lines that are connected to each station and orders them in descending order. This data can be a bit deceiving because "King's Cross St Pancras" is ranked only fifth in the ranking, this station is the one with the most underground lines and someone might argue that underground lines are better than normal train lines for everyday activities.

```

1 MATCH (s:Station)-[r:CONNECTED_BY]->()
2 WITH s, count(DISTINCT r.line) AS num_lines
3 WHERE num_lines > 1
4 RETURN s.name AS Station, num_lines AS NumberOfLines
5 ORDER BY num_lines DESC

```

All the queries that we just saw analyze the bus and train networks by themselves. It can be useful to see the closest train station to a specific bus stop. We can do it in this way by filtering the nodes using the latitude and longitude:

```

1 MATCH (bus:BusStop {name: "WHITGIFT CENTRE"}), (station:Station)
2 WITH bus, station,
3   point({latitude: bus.latitude, longitude: bus.longitude}) AS
   busPoint,
4   point({latitude: station.latitude, longitude: station.longitude})
   AS stationPoint
5 RETURN bus.name AS BusStop,
6   station.name AS NearestStation,

```

```

7         point.distance(busPoint, stationPoint) AS Distance
8 ORDER BY Distance ASC
9 LIMIT 5

```

A variation of this query is this one that finds the 10 bus stops and stations that are closest to each others:

```

1 MATCH (bus:BusStop), (station:Station)
2 WITH bus, station,
3     point({latitude: bus.latitude, longitude: bus.longitude}) AS
      busPoint,
4     point({latitude: station.latitude, longitude: station.longitude})
      AS stationPoint
5 RETURN bus.name AS BusStop, station.name AS NearestStation,
6     point.distance(busPoint, stationPoint) AS Distance
7 ORDER BY Distance ASC
8 LIMIT 10

```

2.2.2. Other queries

These queries are interesting mainly for statistical purposes and they are here to quantify the informations in the dataset.

This query returns the number of stations, bus stops and the average station and stop connections:

```

1 MATCH (bus:BusStop)
2 WITH count(bus) AS NumberOfBusStops, avg(COUNT { (bus)-[:NEXT_STOP
      ]->() }) AS AvgBusStopConnections
3
4 MATCH (station:Station)
5 WITH NumberOfBusStops, AvgBusStopConnections, count(station) AS
      NumberOfStations, avg(COUNT { (station)-[:CONNECTED_BY]->() }) AS
      AvgStationConnections
6
7 RETURN NumberOfBusStops, NumberOfStations, AvgBusStopConnections,
      AvgStationConnections

```

This query allows us to see the number of stops for each bus or train line. the results are ordered by the number of stops in descending order:

```

1 MATCH (station1:Station)-[r:CONNECTED_BY]->(station2:Station)
2 WITH DISTINCT r.line AS Line, collect(DISTINCT station1) AS Stops
3 RETURN Line AS Route, size(Stops) AS StopCount, "Tain or Metro" AS
   Type ORDER BY StopCount DESC
4
5 UNION
6
7 MATCH (busStop1:BusStop)-[r:NEXT_STOP]->(busStop2:BusStop)
8 WITH DISTINCT r.route AS Route, collect(DISTINCT busStop1) AS Stops
9 RETURN Route, size(Stops) AS StopCount, "Bus" AS Type ORDER BY
   StopCount DESC

```

This query returns the average distance between each bus stop and station:

```

1 MATCH (bus:BusStop), (station:Station)
2 WITH bus, station,
3   point({latitude: bus.latitude, longitude: bus.longitude}) AS
   busPoint,
4   point({latitude: station.latitude, longitude: station.longitude})
   AS stationPoint
5 WITH point.distance(busPoint, stationPoint) AS Distance
6 RETURN avg(Distance) AS AverageDistance

```

This last query is here to highlight some anomalies in the dataset, in particular there is a surprising amount of bus stops (315 nodes) that are not connected to any other node, also there are 883 virtual stops (stops that do not have a physical structure to indicate them and were also probably temporary), 31 of these virtual stops have a missing name that we replaced with "Unknown".

```

1 MATCH (bus:BusStop)
2 WHERE NOT (bus)-[:NEXT_STOP]->()
3 RETURN count(bus) AS Count, 'UnconnectedBusStops' AS Type
4
5 UNION
6
7 MATCH (bus:BusStop)
8 WHERE bus.virtual_stop = "1"
9 RETURN count(bus) AS Count, 'VirtualBusStops' AS Type
10
11 UNION

```

```
12  
13 MATCH (bus:BusStop)  
14 WHERE bus.name = "Unknown"  
15 RETURN count(bus) as Count, 'MissingName' AS Type
```

3 | Extra

3.1. Introduction

For the extra we decided to develop two Python notebooks that connected directly to the Databases previously described using the available libraries Python offered.

3.2. Requirements

There are a few simple requirements to be able to run the notebooks:

- *Jupyter*: Jupyter is a web application for creating and sharing computational documents. It is necessary in order to run the notebooks locally and be able to connect to the DataBases.

If you're using modern IDEs such as *Pycharm* or *VSCode* installing Jupyter is as simple as installing a plug-in; if that's not the case you can follow the tutorial at this [link](#).

- You need to have a running DataBase instance including the dataset that have been described in this document.

For the MongoDB part we recommend you to install *MongoDBCompass*, a graphical interface for creating and managing MongoDB DataBases.

For the Neo4j part we recommend you to install *Neo4jDesktop*, a client application that helps you managing Neo4j DataBases.

- The instances of the DataBases must be run on a known URL in order to connect to them using the notebooks. You can also run them remotely, but we recommend to run them locally for ease of use.
- The notebooks should be able to install any Python library used in development by itself. If some errors occur, almost all the Python libraries can be installed using the *pip* installation manager.

3.3. The notebooks

3.3.1. MongoDB

The MongoDB notebook consists of some statistical analysis done extracting data from the *California House Pricing* Dataset used for this project.

The aim of the notebook was showing interesting data through the use of graphs. Some data includes the pricing of the houses respective to its location as well as the population. Lastly the correlation of the various fields of the Dataset was explored and commented.

All the work done and its comments can be found in the Notebook.

3.3.2. Neo4j

The Neo4j notebook aims at exploring the *London Datastore* Dataset, showing a map of the buses and undergrounds of the English capital.

The main focus of the notebook is of developing a path finding algorithm that builds a path that goes from one station (bus or underground) to another.

The challenging part of this algorithm was taking into consideration the fact that not all stops are directly connected to one another; indeed, sometimes a path may require you to cover some distance by foot. In order to achieve this, the fields describing the coordinates of the stations have proven to be very useful, since they allowed setting a maximum distance one can cover by foot.

It must be said that the algorithm is far from perfect, since it aims at finding *one* path and not the best path. It can sure be improved by taking into consideration the time that it takes to travel from one stop to another using the bus or the underground, or by making it choose the path with the smallest number of changes. Although it is not perfect, it can be a starting point in developing a far more efficient and useful path finding algorithm in the London transportation network.

All the work done and its comments can be found in the Notebook.

List of Figures

1.1	Image taken from: California Housing Price Prediction on GitHub	3
1.2	Db and collection creation	6
1.3	Avalon, Santa Catalina Island	11
1.4	Image taken from: California population distribution	13
1.5	Los Angeles, Hollywood	15
1.6	San Francisco	18
1.7	High desert state prison	23
1.8	Santa Barbara	27
1.9	Union Square	30
2.1	Metropolitan line	35

List of Tables

1.1 Dataset Variables with Descriptions	2
---	---

